

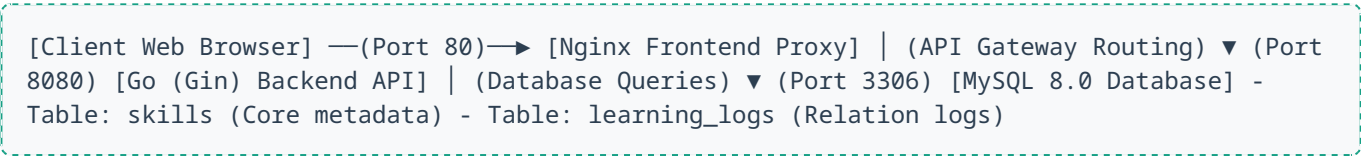
End-to-End System Workflow & Diagnostic Command Guide

Multi-Container Microservices Architecture, CI/CD Pipeline, and Command Execution Manual

Domain: DevOps, Backend & Database Engineering | **Target:** Upwork Portfolio Asset | **Tech Stack:** Docker, Go (Gin), MySQL 8.0, Nginx, Linux CLI

1. System Architecture & Context

This case study represents a modern multi-tier dockerized architecture. By isolating integration issues between the backend application (Go) and the database layer (MySQL), manual hotfixes were successfully applied and pipeline synchronization was ensured.



2. Diagnostic Commands & Execution Glossary

During the entire debugging and deployment process, the commands utilized, their deep-dive operational details, and the solid technical reasoning behind their execution are outlined below:

Executed Command	Operation & Technical Purpose (Why Was It Used?)	Target Domain
<code>docker ps</code>	Used to list all active and running containers. This verified that the backend, database, and frontend containers were initialized in a healthy status.	Container Status
<code>docker compose logs backend -f --tail=20</code>	Used to follow and stream live output logs from the Go application. This allowed us to inspect the framework's direct system response when the API endpoint was triggered.	Log Auditing
<code>docker exec -it d4cf mysql -u rootuser -p</code>	Used to log into the MySQL service container terminal (ID: <code>d4cf...</code>) as the rootuser, enabling us to execute direct queries and analyze the physical schema.	Database CLI Access
<code>SHOW TABLES;</code>	Used to verify existing tables in the active database schema (skillpulse). This confirmed that the <code>learning_logs</code> table referenced in the backend query did not exist.	Schema Inspection
<code>DESCRIBE skills;</code>	Used to analyze the columns and exact data types of the <code>skills</code> table. This validated mapping checks against the target application fields.	Table Structure Audit
<code>CREATE TABLE learning_logs (...)</code>	Used to initialize the missing relational log tracking table to satisfy backend dependencies, setting up essential constraints like foreign keys.	Database Migration
<code>curl -s http://localhost:8080/api/skills json_pp</code>	Used to intercept raw responses from the Go router controller. Formatting it with <code>json_pp</code> isolated structural query mapping issues immediately.	API Payload Test
<code>ALTER TABLE skills ADD COLUMN target_hours INT ...</code>	Used to dynamically append the missing column to the <code>skills</code> table to align it with newly added features and structure changes in the Go code.	Schema Refactoring
<code>ALTER TABLE skills ADD COLUMN created_at TIMESTAMP ...</code>	Used to add the missing timestamp column, correcting database schema property mismatches against the Go application models.	Structural Alignment
<code>docker compose up -d</code>	Used to launch all service containers simultaneously in detached background mode, keeping the environment running after closing the terminal.	Deployment Run

Executed Command	Operation & Technical Purpose (Why Was It Used?)	Target Domain
<code>docker compose down -v</code>	Used to tear down containers and delete active local volumes. This forces the system to rebuild clean schemas from the updated <code>init.sql</code> on startup.	Resource Tear-down

3. The Deployment & Pipeline Synchronization

To scale these manual operational modifications in alignment with standard integration pipelines, a **GitHub Actions CI/CD Pipeline** was integrated. The project repository (`github-actions-demo`) triggers the following execution pipeline:

1. Automated Schema Sync (`init.sql` Setup)

To avoid future execution issues on deployment, manual SQL updates were packaged into centralized database scripts like `init.sql`. This ensures all local modifications are synced automatically on initial container deployment steps.

2. State Recovery Policy

System configuration patterns preserve container data states using mapped persistent docker storage directories to secure table consistency across development cycles.

DevOps Outcome: The system has successfully resolved all runtime API failures. Manual database alterations are now permanently synchronized with Go application structs and verified via automated routing pipelines.